🔹 **Fordeler med OOP (objektorientert programmering)**

OOP handler om å strukturere kode rundt **objekter** i stedet for bare funksjoner. Det gir:

- Mer **oversiktlig kode**
- Enklere **vedlikehold**
- Bedre **gjenbruk**
- Mer **realistisk modellering** av virkeligheten

🔑 **Begrepene**

🔒 **1. Innkapsling (Encapsulation)**

👉 Vi skjuler data og styrer tilgang via metoder.

- Beskytter data mot feil bruk
- Du bestemmer hva som kan endres

```
class Konto:
  def __init__(self, saldo):
    self._saldo = saldo  # "beskyttet"

  def sett_inn(self, belop):
    self._saldo += belop

  def hent_saldo(self):
    return self._saldo
```

💡 Poeng: Man manipulerer ikke _saldo direkte utenfra.

🎭 **2. Abstraksjon (Abstraction)**

👉 Vi skjuler kompleksitet og viser bare det viktigste.

- Brukeren trenger ikke vite *hvordan* noe fungerer
- Kun *hva* det gjør

```
class Kaffemaskin:
  def lag_kaffe(self):
    print("Lager kaffe...")  # masse kompleksitet skjult
```

💡 Du trykker på en knapp → trenger ikke vite hva som skjer inni.

🧬 3. Generalisering (Arv / Inheritance)

👉 Lage en generell klasse som andre kan arve fra.

- Felles egenskaper samles ett sted

```
class Dyr:
  def lag_lyd(self):
    print("Lyd")

class Hund(Dyr):
  def lag_lyd(self):
    print("Voff")
```

💡 Hund arver fra Dyr

♻️ 4. Gjenbruk (Reusability)

👉 Du slipper å skrive samme kode flere ganger.

- Bruk klasser på nytt
- Arv hjelper mye her

```
class Kalkulator:
  def add(self, a, b):
    return a + b
```

💡 Kan brukes overalt i programmet.

🔀 5. Polymorfisme (Polymorphism)

👉 Samme metode kan oppføre seg forskjellig.

- Samme navn → ulik oppførsel

```
dyr = [Hund(), Dyr()]

for d in dyr:
  d.lag_lyd()
```

Output:

```
Voff
Lyd
```

💡 Samme metode (lag_lyd) → ulikt resultat

Kort oppsummert

Begrep	Hva det betyr
Innkapsling	Skjule data, kontrollere tilgang
Abstraksjon	Skjule kompleksitet
Generalisering	Arv fra en baseklasse
Gjenbruk	Bruke kode flere steder
Polymorfisme	Samme metode, ulik oppførsel